Blockchain Implementation Report

Tamper-Evidence, Difficulty vs. Effort, and Design Rationale

Name: Theekshana A.P.K.
Emp.ID : EG/2022/5368
Task: Intern Assessment
Date: 10/07/2026

1. Tamper-Evidence

1.1 Setup


```
    },
    {
      "index": 2,
      "timestamp": 1783578903,
      "txns": [
        {
          "sender": "coinbase",
          "recipient": "miner1",
          "amount": 50
        },
        {
          "sender": "alice",
          "recipient": "kamal",
          "amount": 100
        }
      ],
      "hash": "0004897fa64d36b4308b04d344b39245b89246b09e769245d04698812b7ea85f",
      "nonce": 48,
      "previous_hash": "000d4388160d1d92ba51ef1d99f9e8f02dd26b229a717755ca0a3df6a5359026"
    },
    {
      "index": 3,
      "timestamp": 1783660319,
      "txns": [
        {
          "sender": "coinbase",
          "recipient": "miner2",
          "amount": 30
        },
```


Figure 1: Before alter

1.2 Before Tampering — Validation Output

```
D:\Sublime Text\go_assesment>.\toychain.exe mine --miner miner2 --reward 30
Mined block 3 hash=0009a609d9e644ddad39238b25cbde87362bf9f4e4db2583e7309877f90a5425 nonce=1620

D:\Sublime Text\go_assesment>
```

1.3 The Tamper

Field	Original value	Tampered value
Transaction:coinbase reward to Miner 1	50	40

1.4 After Tampering — Validation Output


```
    },
    {
      "index": 2,
      "timestamp": 1783578903,
      "txns": [
        {
          "sender": "coinbase",
          "recipient": "miner1",
          "amount": 40
        },
        {
          "sender": "alice",
          "recipient": "kamal",
          "amount": 100
        }
      ],
      "hash": "0004897fa64d36b4308b04d344b39245b89246b09e769245d04698812b7ea85f",
      "nonce": 48,
      "previous_hash": "000d4388160d1d92ba51ef1d99f9e8f02dd26b229a717755ca0a3df6a5359026"
    },
    {
      "index": 3,
      "timestamp": 1783660319,
      "txns": [
        {
          "sender": "coinbase",
          "recipient": "miner2",
          "amount": 30
        },
```


Figure 2: After alter

```
D:\Sublime Text\go_assesment>.\toychain tx add --from alice --to kamal --amt 100
Transaction added to pool.

D:\Sublime Text\go_assesment>.\toychain.exe mine --miner miner2 --reward 30
Error: validation failed after mining: tampered hash at block 2
Usage:
  toychain mine [flags]
```

1.5 Why the Chain Becomes Invalid

Tampering with Blocks Transactions.Amount causes two checks in blockchain Validate () to fail in sequence.

1. current. Hash != CalculateHash(current)

In blockchain.Validate(), the loop recomputes each block's hash and checks:

if current.Hash != CalculateHash(current)

Since Transactions is part of that hash input, changing Amount in current.Transactions changes the output of CalculateHash(current). The stored field current.Hash, however, still holds the value computed *before* tampering, so the comparison fails.

2. current.PrevHash ! = prev.Hash

If the tampered block isn't the last one in the chain, the next block still carries the old PrevHash value. Validate() also checks:

if current.PrevHash != prev.Hash

Because block 1's contents changed, its *correct* hash should also change — but nothing recomputed or updated it, so block 1's stored Hash is now stale relative to its own content, and block 2's PrevHash field (which was recorded against the original, untampered block 1 hash) no longer lines up correctly with how block 1 ought to be evaluated.

This breaks the chain link for every block after the tampered one, since each later block's validity depends on the assumption that the previous block's Hash value is exactly what it was when the chain was built.

---

2. Difficulty Versus Effort

2.1 Method

I ran the mining experiment on your implementation using a fixed sample block and these difficulties

2.2 Results

Difficulty (leading zeros)	Time taken (ms)	Hashes tried (nonce count)
1	0	21
2	0	142
3	1.5	1,829
4	28	40,773
5	780	1,249,428

2.3 Trend Analysis

The mining effort increases much faster than linearly as the proof-of-work difficulty increases. This behavior is expected because the mining algorithm repeatedly changes the block's nonce until the SHA-256 hash satisfies the required difficulty target. In this implementation, a valid block hash must begin with a specified number of leading hexadecimal zeros:

strings.Repeat("0", difficulty)

Since CalculateHash() produces a SHA-256 hash encoded as a lowercase hexadecimal string, each additional required leading zero fixes one more hexadecimal digit.

Why the Mining Time Increases: A SHA-256 hash behaves like a random value, where each hexadecimal digit has 16 possible values (0–f). Therefore:

- Difficulty 1: Probability of success = 1/16
- Difficulty 2: Probability of success = $1/16^2$ = 1/256
- Difficulty 3: Probability of success = $1/16^3$ = 1/4,096
- Difficulty 4: Probability of success = $1/16^4$ = 1/65,536
- Difficulty 5: Probability of success = $1/16^5$ = 1/1,048,576

3. Design Write-Up

3.1 Hashing Scheme

My block hash is computed by hashing a deterministic serialization of these fields, in this order:

Order	Field	Why it's included
1	Index	Makes the block position part of the identity.
2	Timestamp	Ensures the creation time affects the hash.
3	Transactions	Included in full so every transfer changes the hash.
4	PrevHash	Links the block explicitly to the previous block.
5	Nonce	Placed last because mining searches by changing only this value.

3.2 Validation and Chain-Wide Integrity

The chain validation process checks every block and the sequence as a whole:

1. Recalculate each block's hash from its stored field values.
   - if stored.Hash != CalculateHash(block), that block is corrupted
2. Verify PrevHash of each block matches the previous block's Hash.
   - this enforces the cryptographic chain linkage
3. Confirm each block satisfies proof-of-work.
   - the hash must begin with the required number of leading zeros (Difficulty)
4. Check index order.
   - every block index must be exactly one greater than the previous block
5. Check timestamp order.
   - each block must have a timestamp ≥ the previous block's timestamp

Why this protects the whole chain

Because each block contains the previous block's hash:

- if any block's data changes, its own hash changes
- that breaks the next block's PrevHash link
- validation then fails on the subsequent block too

So, tampering with one block invalidates every later block in the chain.

4. Discussion Questions

How does the previous-hash link make tampering with an old block impractical in a real chain, even though it is trivial in your local toy?

Every block in this program stores both its own hash (Hash) and the hash of the previous block (PrevHash). If a transaction inside an earlier block is changed, the regenerated hash of that block no longer matches its stored hash. As a result, validation immediately detects a hash mismatch. Furthermore, because the next block stores the original hash in its PrevHash field, the link between the two blocks is also broken, causing validation to fail for subsequent blocks.

In this toy blockchain, I control the entire blockchain locally, so I could modify the block, recompute the hashes, and re-mine all affected blocks. However, in a real blockchain, this is computationally impractical because an attacker would need to recompute the proof-of-work for the modified block and every block after it while simultaneously catching up with and surpassing the honest network. Unless the attacker controls a majority of the network's consensus power, the altered chain would be rejected by other nodes. This chained hashing mechanism is what makes blockchains tamper-evident and resistant to modification.

Proof-of-work is one way to decide who may add the next block. Name at least one alternative and give one advantage and one drawback versus proof-of-work.

One alternative consensus mechanism is **Proof-of-Stake (PoS)**.

Advantage:
Proof-of-Stake consumes significantly less energy than Proof-of-Work because validators are selected based on the amount of cryptocurrency they stake rather than performing computationally intensive mining. This generally allows for faster block production and improved scalability.

Drawback:
Proof-of-Stake may concentrate influence among participants with larger stakes. If not carefully designed, it can also

introduce challenges such as the "nothing-at-stake" problem, where validators have little cost associated with supporting multiple competing chains.

Another possible alternative is **Proof-of-Authority (PoA)**, which is highly efficient for private or permissioned blockchains but relies on trusted authorities, making it more centralized.

List three concrete ways your toy differs from a production blockchain. Pick one and sketch how you would add it.

This toy blockchain is intentionally simplified and differs from production blockchains in several important ways:

1. No distributed consensus: The blockchain runs as a single local process and does not communicate with peer nodes or participate in network-wide consensus.
2. No digital signatures: Transactions are not signed using cryptographic key pairs, so there is no mechanism to verify that the sender is actually authorized to spend funds.
3. No Merkle tree: Transactions are stored directly within each block rather than being summarized using a Merkle tree, which would allow efficient verification of transaction inclusion without downloading the entire block.

Sketch — adding transaction signatures: